ALERT-02: Choosing and Building Detection

> **Series:** ALERT — Alerting Strategy and Design | **Notebook:** 02 of 05 | **Created:** June 2026 | **Last Updated:** 10/14/2026

Overview

Detection is a choice between four mechanisms, and picking the wrong one is the root of most alert noise. This notebook is the **decision framework** — which mechanism for which signal, and why — then it hands off to AIOPS-02 for the build mechanics and SLO-04 for reliability alerting. It deliberately does not re-document the anomaly detector's knobs; it tells you *which* tool to reach for.

Table of Contents

1. The Four Mechanisms
2. The Decision
3. Anti-Patterns
4. Migrating Classic Metric Events: Measure First
5. Prototype Before You Commit
6. Hand-Off

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3. Custom alerts are created in **Settings** from SaaS 1.344, and in the Anomaly Detection app before that (§ 2)
Prior reading	ALERT-01 (the end-to-end picture)
Build mechanics	AIOPS-02 (anomaly detector setup), SLO-02/04 (SLIs and burn-rate)

1. The Four Mechanisms

Mechanism	What it detects	Owns the detail
OOTB Davis	Latency/error/saturation anomalies, automatically, with seasonal baselines	AIOPS-02 §1
Custom Davis anomaly detector	A business-specific signal Davis does not cover, expressed as a DQL query an analyzer judges	AIOPS-02 §4
OpenPipeline-derived metric	A signal that lives in logs/spans, extracted to a metric at ingest and then alerted on cheaply	OPIPE, FAQ-09
SLO burn-rate	A user journey burning its error budget too fast	SLO-04

These are not alternatives to rank once — they are a toolkit. Most environments use all four.

2. The Decision

Walk it top-down and stop at the first match:

1. **Is OOTB Davis already detecting it?** Check the problem feed and anomaly detection settings first. If yes → tune sensitivity, do not build anything.
2. **Is it a reliability promise about a user journey?** → an **SLO** with burn-rate alerting (SLO-04). This is the highest-signal option.

3. **Does the signal live in logs or spans, and recur?** → extract an **OpenPipeline metric**, then put a detector on the metric. Cheaper than querying raw data on every evaluation.
4. **Is it a custom condition on existing metrics?** → a **custom Davis anomaly detector** with an auto-adaptive or seasonal analyzer.

The cost — to build and to maintain — rises as you go down. Staying high is the lever on noise (ALERT-01 §2).

> **Where a custom detector is created (SaaS 1.344+).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344's staged rollout started 07/29/2026, so Settings is the place to create a custom detector. On a tenant still below 1.344, the **Anomaly Detection** app is where custom alerts are created, and AIOPS-02 § 4 walks that build.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*, SaaS 1.344 release notes (DT docs) — *"Rollout start on Jul 29, 2026"*.

3. Anti-Patterns

- **Static thresholds on traffic-correlated metrics.** They alert every off-peak hour and every traffic spike. Use auto-adaptive or seasonal (AIOPS-02 §1). Reserve static thresholds for true hard limits (SLO/contract/capacity).
- **Duplicating Davis.** Before building a custom detector, confirm OOTB Davis or an existing metric event does not already cover the condition. Duplicate detection means duplicate alerts.
- **Querying logs/traces directly in a recurring detector.** Pays query cost on every evaluation, forever. Extract a metric first (FAQ-09, OPIPE).
- **A detector with a bare event template.** No team or area property means nothing to route on (ALERT-01 §4). Set `dt.smartscape_source.id` too. The docs ask for *"an existing Smartscape entity ID, like a host or service entity ID rather than an arbitrary string"*, because that is what lets the correlation engine resolve the entity and apply its same-entity grouping rule. They do not say what happens to an event that has no source, but on a real tenant (7 days to 10/06/2026) every custom alert raised without one, 670 events, was attached to the environment entity alone. Problems attached

only to the environment averaged 8.5 grouped events each, against about one for every other problem: many alerts end up in one problem whose only affected entity is the environment, which points no one at the thing that broke. Treat that as the likely outcome, not a documented rule. Either way, no threshold change fixes it; only the event template does (AIOPS-02 §4).

- **Splitting the alert on a high-cardinality dimension.** Grouping a detector by application version, pod name, or HTTP status code turns one condition into one alert per dimension value. Alert volume then scales with your deployment frequency, and alerts strand themselves on entities that no longer exist — a pod replaced an hour ago still carries an open problem. Alert on the aggregate; working out *which* version or pod caused it is a downstream investigation step, not an alerting dimension. (Davis's own multi-dimensional baselining is a different mechanism and does want the dimensions — AIOPS-02 §1.4.)
- **The cloned detector.** In a fast rollout the dominant noise source is not one badly-tuned alert — it is a base detector template copied across teams with its threshold, sensitivity, and routing left unchanged. Two weeks later that team has muted the channel. You cannot review your way out of this one clone at a time; defend at the template, not at each copy.

Make the base template safe by default. The fix is to make the *default* shape adaptive, not static, so the worst case of a copy-paste is a correctly-scoped, loosely-tuned alert — never an unscoped firehose. A base detector template should ship with:

- An **auto-adaptive (or seasonal) analyzer as the default**, not a static threshold — the only inputs a team must supply are the metric/selector and the owning team/area property.
- **No-data does not alert**, so missing telemetry during onboarding stays silent until instrumentation lands.
- A **violating-samples / sliding-window minimum** (e.g. ≥3-of-5) so a single transient spike never pages.
- **Routing bound to the team/area property the template already requires**, so even an untuned clone reaches the right owner.

A second lever, at a different layer

Available (SaaS 1.344): the workflow **Problem trigger** has a **Minimum duration** option (under *Advanced options*) that postpones the trigger until the problem has been open for at least the configured duration: 5 minutes up to one week. SaaS 1.344's rollout started **07/29/2026**; verify it has reached your tenant before designing around it.

This is **not** a fifth analyzer knob, and it does not replace the sliding-window minimum above. The analyzer parameters decide whether a series is anomalous; Minimum duration decides how long a problem must stay open before the workflow runs. The problem itself opens immediately. They act on different steps, so they compose — and because the delay applies at the workflow trigger, it damps notifications from flapping detectors you do not own, which is exactly the case a template standard cannot reach. Where it is not available yet, the violating-samples / sliding-window minimum remains the control to rely on for transient-spike noise.

> **Sources:** SaaS 1.344 release notes (DT docs), Event triggers for workflows (DT docs) — *"The Minimum duration option postpones the trigger until the problem has been open for at least the configured duration."* Avoid overalerting (DT docs) — *"an existing Smartscape entity ID, like a host or service entity ID rather than an arbitrary string"*; it does not describe events without a source, so the environment-only figures are tenant measurements (`dt.davis.events` and `dt.davis.problems` , 7 days, run 10/06/2026). **Derived:** the "composes rather than substitutes" placement follows from the delay acting on the problem-to-notification step while analyzer parameters act on the series.

4. Migrating Classic Metric Events: Measure First

If you are moving off classic alerting, the decision above applies to every classic metric event you own, not just to new signals. The mistake to avoid is porting them all one-to-one. A tenant accumulates metric events over years: some were created by an extension or a cloud integration, some guarded a system that is gone, and some still matter. The configuration list cannot tell those apart. The alert history can.

First, what is configured. Classic metric events are Settings 2.0 objects under `builtin:anomaly-detection.metric-events` , so one read lists them, with the enabled flag and the query type:

```
dtctl get settings --schema builtin:anomaly-detection.metric-events -o json --limit 0
```

The query type matters because the Anomaly Detection app's transformation only covers one of the two kinds:

> *"You can only transform metric selectors."*

Events built on a plain metric key (`METRIC_KEY`) cannot go through the transformation and have to be rebuilt by hand. When a transformation does run, *"the metric event we selected for transformations is automatically disabled, and the newly created configuration is active instead."* Before you transform an event, decide whether you still want it.

A zone-scoped metric event loses its zone on transform. The custom-alert page says *"Management zones aren't supported in Anomaly Detection and will be lost after transforming metric events."* and *"The converted metric event is automatically disabled, and the newly created configuration is active instead."* Taken together, one click swaps a detector that watched one zone for an active detector that evaluates its metric selector across the whole environment — expect it to fire for entities the zone used to exclude. Before you rely on the transformed alert, set its **segment** scope (*"Segment filters are automatically applied to your query during execution."*) and preview the result. MZ2POL-05 builds the segments; MZ2POL-15 finds which metric events name a zone.

Then, what actually fires. Every alert a metric event raises lands in `dt.davis.events` carrying the settings object it came from, so the history joins back to the configuration on `dt.settings.object_id` :

```
// Which classic metric events raised anything in the last 30 days?
// Filter on the settings schema, NOT on event.provider: "METRIC_EVENTS" is
also the
// provider for the new DQL anomaly detectors and for built-in infrastructure
detection.
fetch dt.davis.events, from:-30d
| filter dt.settings.schema_id == "builtin:anomaly-detection.metric-events"
| summarize {alerts = count(), last_raised = max(timestamp)}, by:
{dt.settings.object_id, event.name}
| sort alerts desc
```

Compare the two lists. An enabled metric event that does not appear in the history has raised nothing in the window.

What this looked like on a real tenant (09/29/2026). 125 classic metric events were configured and 19 were enabled: 18 on a metric key and 1 on a metric selector. The query above returned **zero rows**. Over the same 30 days, the identical query with the schema set to `builtin:davis.anomaly-detectors` returned 35 detectors, which rules out an empty result caused by the query itself. So the whole migration came down to 19

decisions. Of those, 18 could not be transformed, and not one had raised an alert in a month.

Two traps.

- `event.provider == "METRIC_EVENTS"` **is not "classic metric events".** On the same tenant that provider covered 1,054 events from `builtin:anomaly-detection.infrastructure-hosts`, 500 from the new `builtin:davis.anomaly-detectors`, and a few from `builtin:anomaly-detection.infrastructure-disks`, but none from classic metric events. A migration inventory built on the provider mostly counts detectors you have already migrated.
- **Silence is not the same as useless.** A metric event that guards a rare condition, such as a certificate about to expire or a batch job that failed, may stay quiet for months and still be the only thing watching that condition. Widen the window, check the event's name and owner, and make a *retire* or *migrate* decision per event. Do not delete everything that did not fire.

For each event you keep, run it through the decision above. A static threshold is where the classic event started, and it is not necessarily where it should end up.

> **Sources:** Upgrade Metric Alerting (DT docs) — both quotes above; Configure a simple custom alert (DT docs) — *"Management zones aren't supported in Anomaly Detection and will be lost after transforming metric events."*, *"The converted metric event is automatically disabled, and the newly created configuration is active instead."*; Use segments with Anomaly Detection custom alerts (DT docs) — *"Segment filters are automatically applied to your query during execution."* **Derived:** a zone-scoped metric event widens to the whole environment on transform unless a segment is set — combines the two custom-alert quotes. **Dictionary:** `dt.settings.schema_id` (`experimental`), `dt.settings.object_id` (`experimental`), `event.provider` (`stable`), read 09/29/2026. Tenant counts from `dtctl get settings` and the query above, both run 09/29/2026, with the `builtin:davis.anomaly-detectors` run as the control. Re-run 10/06/2026 as shown: zero rows and no notifications, while the control returned 2,434 events from 8 detectors.

5. Prototype Before You Commit

Whichever mechanism you choose, develop the underlying query in a notebook first — run it, confirm it returns a sane result over a representative window, *then* wire it into the

detector or SLO. A detector built on a query that silently returns nothing never fires and is worse than no alert, because the team believes it is covered. This notebook-as-scratchpad discipline is covered in AIOPS-02 §4 and SLO-02 §6.

Then deploy relaxed, and tighten later. Once the query is sound, resist configuring the detector at the sensitivity you think you want. Ship it with a high static threshold or a wide deviation margin, watch it for one to two weeks without acting on every trigger, and only then reduce the margin toward the real signal.

The asymmetry is the whole argument: under-alerting for a fortnight is recoverable, whereas a team that muted your channel because the first week was noise is a much harder problem — and they will still be muted when the detector is finally tuned correctly. Alert fatigue is easier to avoid than to undo.

The rigorous version of this makes the observation window explicit rather than informal: deploy the detector with its event type set to `CUSTOM_INFO`, so it is recorded and queryable but never opens a problem, measure how often it actually fires over two to four weeks, and promote it to `CUSTOM_ALERT` only once that number looks sane. AIOPS-02 §4 covers the mechanics and §8 the measuring query.

> **Sources:** Avoid overalerting (DT docs).

6. Hand-Off

Mechanism chosen	Build it in
Tune OOTB Davis	AIOPS-02 §1–§3
Custom anomaly detector	AIOPS-02 §4 (analyzer, tuning knobs, event template)
Metric events / schemas as code	AIOPS-02 §6, AUTOM-05/06
OpenPipeline-derived metric	OPIPE, FAQ-09
SLO burn-rate	SLO-02 (SLI), SLO-04 (alerting)

Then return to ALERT-03 to route what fires.

> **Sources:** Anomaly Detection app (DT docs). **Derived:** the top-down decision order synthesises

> OOTB-first guidance with the query-cost economics in OPIPE/FINOPS.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.